

ブラウザだけで学ぶ経済・経営統計

JavaScript で実践する回帰分析と仮説検定

mathjs & simple-statistics による実践ガイド

河合勝彦

名古屋市立大学大学院経済学研究科

kkawai@econ.nagoya-cu.ac.jp

2026 年 1 月 8 日

概要

本ガイドは、JupyterLite の JavaScript カーネル上で `mathjs` と `simple-statistics` ライブラリを用いた数学・統計学の学習を支援するものです。全 6 章のノートブックを通じて、基礎的な数学計算から高度な統計解析まで段階的に学習できます。対象読者は社会科学系の学生を想定しており、プログラミングと数学・統計学の両方のスキルを同時に習得することを目標としています。

Jupyter ノートブック：本ガイドの内容を実際に手を動かして学べる Jupyter ノートブック（全 6 章）を GitHub で公開しています。

<https://github.com/kkawailab/kklab-js-math-stats-samples>

動作環境：JupyterLite（JavaScript カーネル対応版）。モダンな Web ブラウザ（Chrome, Firefox, Edge, Safari）で動作します。ライブラリは CDN（esm.sh）からネットワーク経由で読み込むため、インターネット接続が必要です。

指導者向け注意：JupyterLite で JavaScript カーネルを利用するには、`xeus-javascript` を組み込んだ環境を事前にビルドする必要があります。詳細は <https://github.com/jupyter-xeus/xeus-javascript> を参照してください。

キーワード：JupyterLite, JavaScript, 統計学, 回帰分析, 仮説検定, mathjs, simple-statistics, データサイエンス教育

目次

1	本教材の概要	4
1.1	学習目標	4
1.2	使用ライブラリ	4
1.3	ライブラリの読み込み方法	4
2	第 1 章：環境セットアップと基礎	6
2.1	章の目的	6
2.2	主要な概念	6
2.3	mathjs の関数一覧	6
2.4	simple-statistics の関数一覧（基礎）	6

目次		2
2.5	コード例：基本的なデータ分析	7
2.6	練習問題	7
3	第 2 章：線形代数	9
3.1	章の目的	9
3.2	主要な概念	9
3.3	mathjs の関数一覧（線形代数）	9
3.4	コード例：連立方程式の解法	9
3.5	コード例：固有値と固有ベクトル	10
3.6	練習問題	10
4	第 3 章：統計学基礎	12
4.1	章の目的	12
4.2	主要な概念	12
4.3	simple-statistics の関数一覧（統計）	12
4.4	コード例：相関分析	12
4.5	相関係数の解釈	13
4.6	練習問題	13
5	第 4 章：確率分布	15
5.1	章の目的	15
5.2	主要な概念	15
5.3	重要な確率分布の公式	15
5.4	simple-statistics の関数一覧（確率）	15
5.5	コード例：正規分布の確率計算	16
5.6	正規分布の重要な確率	16
5.7	練習問題	16
6	第 5 章：回帰分析	18
6.1	章の目的	18
6.2	主要な概念	18
6.3	simple-statistics の関数一覧（回帰）	18
6.4	最小二乗法の公式	18
6.5	コード例：単回帰分析	18
6.6	コード例：重回帰分析（行列演算）	19
6.7	練習問題	19
7	第 6 章：高度な統計解析	22
7.1	章の目的	22
7.2	主要な概念	22
7.3	仮説検定の手順	22
7.4	simple-statistics の関数一覧（検定・クラスタリング）	22

7.5	参考：クラスタリングのコード例	23
7.6	コード例：2 標本 t 検定（等分散：Student）	23
7.7	コード例：カイ二乗適合度検定	24
7.8	練習問題	24
8	関数リファレンス（総合）	27
8.1	mathjs 関数一覧	27
8.2	simple-statistics 関数一覧	27
9	学習のヒント	29
9.1	効果的な学習方法	29
9.2	よくあるエラーと対処法	29
9.3	発展学習	29
10	本教材の実行環境	30
10.1	JupyterLite とは	30
10.2	Xeus とは	30
10.3	xeus-javascript について	31
10.4	なぜ JavaScript を使うのか	31
11	参考サイト	32
11.1	使用ライブラリ	32
11.2	JavaScript 学習	32
11.3	数学・統計学	32
11.4	JupyterLite	32

1 本教材の概要

1.1 学習目標

本教材を通じて、以下のスキルを習得することを目指します：

1. JavaScript を用いた数学計算の基本操作
2. ベクトル・行列演算と線形代数の応用
3. 記述統計量の計算とデータの要約
4. 確率分布の理解と応用
5. 回帰分析によるデータモデリング
6. 仮説検定とデータに基づく意思決定

1.2 使用ライブラリ

1.2.1 mathjs (バージョン 13.0.0)

公式サイト：<https://mathjs.org/>

包括的な数学ライブラリで、以下の機能を提供します：

- 四則演算、数学関数、三角関数
- 行列・ベクトル演算
- 数式パーサー
- 複素数、単位変換

1.2.2 simple-statistics (バージョン 7.8.8)

公式サイト：<https://simple-statistics.github.io/>

軽量な統計ライブラリで、以下の機能を提供します：

- 代表値・散布度の計算
- 回帰分析
- 確率分布関数
- 仮説検定

1.3 ライブラリの読み込み方法

JupyterLite 環境では、CDN から ESM (ES Modules) として動的にライブラリを読み込みます。JavaScript カーネルでは `await import()` をそのまま利用できます (CDN にアクセスできない環境では読み込みに失敗します)。

Listing 1 ライブラリのインポート

```
1 // mathjs の読み込み (ESM)
2 const math = await import("https://esm.sh/mathjs@13.0.0");
3
4 // simple-statistics の読み込み (ESM)
5 const ss = await import("https://esm.sh/simple-statistics@7.8.8");
```

注意：URL 中の `simple-statistics` に空白が入ると読み込みエラーになります。上記のように正確に記述してください。

2 第1章：環境セットアップと基礎

2.1 章の目的

JavaScript 環境で数学計算を行うための基盤を構築します。mathjs と simple-statistics の基本的な使い方を習得し、簡単なデータ分析ができるようになることを目指します。

2.2 主要な概念

CDN (Content Delivery Network) 外部ライブラリをネットワーク経由で読み込む仕組み

ESM (ES Modules) JavaScript の標準的なモジュールシステム

動的インポート `await import()` を用いた非同期のモジュール読み込み

2.3 mathjs の関数一覧

関数	説明	例
<code>math.add(a, b)</code>	加算	<code>math.add(10, 5) → 15</code>
<code>math.subtract(a, b)</code>	減算	<code>math.subtract(10, 5) → 5</code>
<code>math.multiply(a, b)</code>	乗算	<code>math.multiply(10, 5) → 50</code>
<code>math.divide(a, b)</code>	除算	<code>math.divide(10, 5) → 2</code>
<code>math.sqrt(x)</code>	平方根	<code>math.sqrt(16) → 4</code>
<code>math.pow(x, n)</code>	べき乗	<code>math.pow(2, 10) → 1024</code>
<code>math.log10(x)</code>	常用対数	<code>math.log10(100) → 2</code>
<code>math.log(x)</code>	自然対数	<code>math.log(math.e) → 1</code>
<code>math.abs(x)</code>	絶対値	<code>math.abs(-5) → 5</code>
<code>math.factorial(n)</code>	階乗	<code>math.factorial(5) → 120</code>
<code>math.sin(x)</code>	正弦 (ラジアン)	<code>math.sin(math.pi/2) → 1</code>
<code>math.cos(x)</code>	余弦 (ラジアン)	<code>math.cos(0) → 1</code>
<code>math.tan(x)</code>	正接 (ラジアン)	<code>math.tan(math.pi/4) → 1</code>
<code>math.evaluate(expr)</code>	数式の評価	<code>math.evaluate('2+3*4') → 14</code>

2.4 simple-statistics の関数一覧 (基礎)

関数	説明	数式
<code>ss.mean(arr)</code>	算術平均	$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$
<code>ss.median(arr)</code>	中央値	順序統計量の中央
<code>ss.mode(arr)</code>	最頻値	最も頻度の高い値
<code>ss.min(arr)</code>	最小値	$\min(x_1, \dots, x_n)$
<code>ss.max(arr)</code>	最大値	$\max(x_1, \dots, x_n)$

<code>ss.variance(arr)</code>	母分散	$\sigma^2 = \frac{1}{n} \sum (x_i - \bar{x})^2$
<code>ss.sampleVariance(arr)</code>	標本分散	$s^2 = \frac{1}{n-1} \sum (x_i - \bar{x})^2$
<code>ss.standardDeviation(arr)</code>	母標準偏差	$\sigma = \sqrt{\sigma^2}$
<code>ss.sampleStandardDeviation(arr)</code>	標本標準偏差	$s = \sqrt{s^2}$
<code>ss.quantile(arr, p)</code>	パーセンタイル	p -分位数
<code>ss.interquartileRange(arr)</code>	四分位範囲	$IQR = Q_3 - Q_1$

母集団と標本の区別：`ss.variance` と `ss.standardDeviation` は母分散・母標準偏差（ n で除算）を計算します。標本から母集団を推定する場合は `ss.sampleVariance` と `ss.sampleStandardDeviation`（ $n-1$ で除算、不偏推定量）を使用してください。

2.5 コード例：基本的なデータ分析

Listing 2 データ分析の基本フロー

```

1 // サンプルデータ
2 const data = [78, 85, 92, 65, 88, 72, 95, 80, 77, 83];
3
4 // 代表値の計算
5 console.log('平均:', ss.mean(data));           // 81.5
6 console.log('中央値:', ss.median(data));        // 81.5
7 console.log('最頻値:', ss.mode(data));          // undefined（重複なし）
8
9 // 散布度の計算
10 console.log('標準偏差:', ss.standardDeviation(data));
11 console.log('分散:', ss.variance(data));
12
13 // 四分位数
14 console.log('Q1:', ss.quantile(data, 0.25));
15 console.log('Q3:', ss.quantile(data, 0.75));
16 console.log('IQR:', ss.interquartileRange(data));

```

2.6 練習問題

2.6.1 練習1：基本計算

mathjs を使って以下を計算してください：

1. 144 の平方根
2. 2^8 （2 の 8 乗）
3. 1000 の常用対数（ $\log_{10}$ ）

模範解答

Listing 3 練習1の解答

```

1 // 1. 144 の平方根
2 console.log('144の平方根:', math.sqrt(144)); // 12

```

```
3
4 // 2. 2の8乗
5 console.log('2^8 =', math.pow(2, 8)); // 256
6
7 // 3. 1000の常用対数
8 console.log('log10(1000) =', math.log10(1000)); // 3
```

2.6.2 練習2：統計量の計算

以下のデータについて、平均、中央値、標準偏差を計算してください：

[12, 15, 18, 22, 25, 28, 30, 35, 40, 45]

模範解答

Listing 4 練習2の解答

```
1 const data = [12, 15, 18, 22, 25, 28, 30, 35, 40, 45];
2
3 console.log('平均:', ss.mean(data)); // 27
4 console.log('中央値:', ss.median(data)); // 26.5
5 console.log('標準偏差:', ss.standardDeviation(data).toFixed(4)); // 10.4403
```


3 第2章：線形代数

3.1 章の目的

ベクトルと行列の基本操作を習得し、連立方程式の解法や固有値問題など、線形代数の応用を学びます。

3.2 主要な概念

ベクトル 大きさと方向を持つ量。配列として表現

行列 数値を長方形に配列したもの。2次元配列として表現

内積（ドット積） $\mathbf{u} \cdot \mathbf{v} = \sum_i u_i v_i$

外積（クロス積） 3次元ベクトルに対して定義される演算

ノルム ベクトルの「大きさ」を表す指標

行列式 正方行列に対して定義されるスカラー値 $\det(A)$

逆行列 $AA^{-1} = I$ を満たす行列

固有値・固有ベクトル $A\mathbf{v} = \lambda\mathbf{v}$ を満たす λ と $\mathbf{v}$

3.3 mathjs の関数一覧（線形代数）

関数	説明	用途
<code>math.matrix(arr)</code>	行列オブジェクト作成	行列演算の準備
<code>math.dot(u, v)</code>	内積	類似度、射影
<code>math.cross(u, v)</code>	外積	法線ベクトル
<code>math.norm(v)</code>	ノルム	ベクトルの大きさ
<code>math.transpose(A)</code>	転置	A^T
<code>math.det(A)</code>	行列式	可逆性の判定
<code>math.inv(A)</code>	逆行列	A^{-1}
<code>math.identity(n)</code>	単位行列	I_n
<code>math.zeros(m, n)</code>	ゼロ行列	初期化
<code>math.ones(m, n)</code>	全1行列	初期化
<code>math.diag(arr)</code>	対角行列	対角成分の設定
<code>math.lusolve(A, b)</code>	連立方程式	$A\mathbf{x} = \mathbf{b}$
<code>math.lup(A)</code>	LU 分解	行列分解
<code>math.qr(A)</code>	QR 分解	行列分解
<code>math.eigs(A)</code>	固有値分解	主成分分析など

3.4 コード例：連立方程式の解法

連立方程式 $A\mathbf{x} = \mathbf{b}$ を解く例：

$$\begin{cases} 2x + 3y = 8 \\ 4x + 5y = 14 \end{cases}$$

Listing 5 連立方程式の解法

```

1  const A = [[2, 3], [4, 5]]; // 係数行列
2  const b = [8, 14];          // 定数ベクトル
3
4  // 逆行列を使った解法: x = A^(-1) * b
5  const solution = math.multiply(math.inv(A), b);
6  console.log('解:', solution); // [1, 2]
7
8  // lusolve を使った解法 (推奨)
9  const sol = math.lusolve(A, [[8], [14]]);
10 console.log('x =', sol[0][0], ', y =', sol[1][0]);

```

3.5 コード例：固有値と固有ベクトル

Listing 6 固有値分解

```

1  const M = [[4, 2], [1, 3]];
2  const eig = math.eigs(M);
3
4  console.log('固有値:', eig.values);
5  // [2, 5]
6
7  console.log('固有ベクトル:');
8  eig.eigenvectors.forEach((ev, i) => {
9      console.log('λ ' + (i+1) + ':', ev.vector);
10 });

```

注意: `math.eigs` が返す固有値は、絶対値の昇順で並んでいます。主成分分析などで最大固有値が必要な場合は、配列の最後の要素を参照してください。

3.6 練習問題

3.6.1 練習1：ベクトル演算

ベクトル $\mathbf{a} = [2, 3, 4]$ と $\mathbf{b} = [1, -1, 2]$ について：

1. $\mathbf{a}$ と $\mathbf{b}$ の内積を求めてください
2. $\mathbf{a}$ と $\mathbf{b}$ の外積を求めてください
3. $\mathbf{a}$ のノルム（大きさ）を求めてください

模範解答

Listing 7 練習1の解答

```

1  const a = [2, 3, 4];

```

```

2  const b = [1, -1, 2];
3
4  // 1. 内積
5  const dotProduct = math.dot(a, b);
6  console.log('内積 a・b =', dotProduct);
7  // 計算: 2*1 + 3*(-1) + 4*2 = 2 - 3 + 8 = 7
8
9  // 2. 外積
10 const crossProduct = math.cross(a, b);
11 console.log('外積 a×b =', crossProduct);
12 // [3*2 - 4*(-1), 4*1 - 2*2, 2*(-1) - 3*1] = [10, 0, -5]
13
14 // 3. ノルム
15 const normA = math.norm(a);
16 console.log('||a|| =', normA.toFixed(4));
17 // sqrt(4 + 9 + 16) = sqrt(29) ≈ 5.3852

```

3.6.2 練習2：連立方程式

以下の連立方程式を解いてください：

$$\begin{cases} 3x + 2y = 7 \\ x + 4y = 9 \end{cases}$$

模範解答

Listing 8 練習2の解答

```

1  const A = [[3, 2], [1, 4]]; // 係数行列
2  const b = [[7], [9]];      // 定数ベクトル
3
4  // 方法1: 逆行列を使う
5  const solution = math.multiply(math.inv(A), [7, 9]);
6  console.log('解:', solution); // [1, 2]
7
8  // 方法2: lusolveを使う (推奨)
9  const sol = math.lusolve(A, b);
10 console.log('x =', sol[0][0]); // 1
11 console.log('y =', sol[1][0]); // 2
12
13 // 検算: 3(1) + 2(2) = 7, 1(1) + 4(2) = 9 ✓

```

4 第3章：統計学基礎

4.1 章の目的

記述統計の各種指標を理解し、データの特徴を適切に要約・解釈できるようになることを目指します。

4.2 主要な概念

代表値 データ全体を1つの値で表現（平均、中央値、最頻値）

散布度 データのばらつきを表す指標（分散、標準偏差、範囲）

歪度 分布の非対称性を表す指標（Skewness）

尖度 分布の尖り具合を表す指標（Kurtosis）

共分散 2変数の同時変動を表す指標

相関係数 2変数の線形関係の強さ（ $-1 \leq r \leq 1$ ）

決定係数 説明変数による変動の説明割合（ $R^2 = r^2$ ）

Zスコア 標準化された得点 $z = \frac{x - \mu}{\sigma}$

4.3 simple-statistics の関数一覧（統計）

関数	説明	数式
ss.geometricMean(arr)	幾何平均	$\sqrt[n]{\prod x_i}$
ss.harmonicMean(arr)	調和平均	$\frac{n}{\sum \frac{1}{x_i}}$
ss.rootMeanSquare(arr)	二乗平均平方根	$\sqrt{\frac{1}{n} \sum x_i^2}$
ss.sampleSkewness(arr)	歪度	分布の非対称性
ss.sampleKurtosis(arr)	尖度	分布の尖り
ss.sampleCovariance(x, y)	共分散	$\text{Cov}(X, Y)$
ss.sampleCorrelation(x, y)	相関係数	r_{xy}
ss.zScore(x, mean, std)	Zスコア	$(x - \mu)/\sigma$

4.4 コード例：相関分析

Listing 9 相関係数と決定係数

```
1 const studyHours = [2, 3, 4, 5, 6, 7, 8];
2 const examScores = [55, 60, 68, 75, 82, 88, 95];
3
4 // 相関係数
5 const r = ss.sampleCorrelation(studyHours, examScores);
6 console.log('相関係数 r:', r.toFixed(4)); // 0.9992
7
8 // 決定係数
```

```

9  const rSquared = r * r;
10 console.log('決定係数 R^2:', rSquared.toFixed(4)); // 0.9983
11
12 // 解釈：勉強時間で点数の変動の99.8%を説明できる

```

4.5 相関係数の解釈

$ r $ の範囲	解釈
$0.7 \leq r $	強い相関
$0.4 \leq r < 0.7$	中程度の相関
$0.2 \leq r < 0.4$	弱い相関
$ r < 0.2$	ほとんど相関なし

注意事項：

- 上記の閾値はあくまで目安であり、分野によって基準が異なります
- 相関は因果を意味しない：強い相関があっても因果関係があるとは限りません
- 外れ値、非線形関係、データ範囲の制限により、相関係数の値は大きく変わることがあります

4.6 練習問題

4.6.1 練習1：代表値の計算

以下のデータについて、平均、中央値、最頻値を計算してください：

[5, 7, 7, 8, 9, 10, 10, 10, 12, 15]

模範解答

Listing 10 練習1の解答

```

1  const data = [5, 7, 7, 8, 9, 10, 10, 10, 12, 15];
2
3  console.log('平均:', ss.mean(data)); // 9.3
4  console.log('中央値:', ss.median(data)); // 9.5
5  console.log('最頻値:', ss.mode(data)); // 10
6  // 10が3回出現し、最も頻度が高い

```

4.6.2 練習2：相関係数

以下のデータの相関係数を計算し、相関の強さを解釈してください：

x = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

y = [2.1, 3.9, 6.2, 7.8, 10.1, 12.0, 14.2, 15.9, 18.1, 20.0]

模範解答

Listing 11 練習2の解答

```

1  const x = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];
2  const y = [2.1, 3.9, 6.2, 7.8, 10.1, 12.0, 14.2, 15.9, 18.1, 20.0];

```

```
3
4  const r = ss.sampleCorrelation(x, y);
5  console.log('相関係数 r:', r.toFixed(4)); // 0.9998
6
7  // 決定係数
8  const rSquared = r * r;
9  console.log('決定係数 R^2:', rSquared.toFixed(4)); // 0.9996
10
11 // 解釈: |r| = 0.9998 ≥ 0.7 なので「強い正の相関」がある
12 // xが増加するとyも増加する傾向が非常に強い
```

5 第4章：確率分布

5.1 章の目的

離散・連続確率分布の概念を理解し、確率計算や区間推定ができるようになることを目指します。

5.2 主要な概念

確率質量関数 (PMF) 離散確率変数の確率を与える関数

確率密度関数 (PDF) 連続確率変数の確率密度を与える関数

累積分布関数 (CDF) $F(x) = P(X \leq x)$

二項分布 n 回の試行で成功回数が従う分布 $B(n, p)$

ポアソン分布 単位時間あたりの事象発生回数 $Po(\lambda)$

正規分布 最も基本的な連続分布 $N(\mu, \sigma^2)$

t 分布 小標本での母平均推定に使用

中心極限定理 標本平均は正規分布に収束する

5.3 重要な確率分布の公式

5.3.1 二項分布 $B(n, p)$

$$P(X = k) = \binom{n}{k} p^k (1 - p)^{n-k}$$

期待値: $E[X] = np$ 、分散: $\text{Var}(X) = np(1 - p)$

5.3.2 ポアソン分布 $Po(\lambda)$

$$P(X = k) = \frac{\lambda^k e^{-\lambda}}{k!}$$

期待値: $E[X] = \lambda$ 、分散: $\text{Var}(X) = \lambda$

5.3.3 正規分布 $N(\mu, \sigma^2)$

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right)$$

5.4 simple-statistics の関数一覧 (確率)

関数	説明
<code>ss.cumulativeStdNormalProbability(z)</code>	標準正規分布の CDF $\Phi(z)$
<code>ss.probit(p)</code>	標準正規分布の分位点 $\Phi^{-1}(p)$
<code>ss.errorFunction(x)</code>	誤差関数 $\text{erf}(x)$

<code>math.combinations(n, k)</code>	組み合わせ $\binom{n}{k}$
<code>math.factorial(n)</code>	階乗 $n!$

5.5 コード例：正規分布の確率計算

Listing 12 正規分布の活用

```

1 // テストの点数が平均70点、標準偏差10点の正規分布に従う
2 const mu = 70, sigma = 10;
3
4 // 80点以上を取る確率
5 const z80 = (80 - mu) / sigma; // z = 1
6 const pAbove80 = 1 - ss.cumulativeStdNormalProbability(z80);
7 console.log('80点以上の確率:', (pAbove80 * 100).toFixed(2) + '%');
8 // → 15.87%
9
10 // 上位10%のライン
11 const z90 = ss.probit(0.90); // 90パーセンタイル
12 const score90 = mu + z90 * sigma;
13 console.log('上位10%ライン:', score90.toFixed(1) + '点');
14 // → 82.8点

```

5.6 正規分布の重要な確率

範囲	確率
$\mu \pm 1\sigma$	68.27%
$\mu \pm 2\sigma$	95.45%
$\mu \pm 3\sigma$	99.73%

5.7 練習問題

5.7.1 練習1：二項分布

合格率60%の試験を10人が受験した場合、7人以上合格する確率を求めてください。

模範解答

Listing 13 練習1の解答

```

1 // 二項分布の確率質量関数
2 function binomialPMF(k, n, p) {
3     return math.combinations(n, k) * Math.pow(p, k) * Math.pow(1 - p, n - k);
4 }
5
6 const n = 10; // 試行回数
7 const p = 0.6; // 成功確率
8

```



```
9 // P(X >= 7) = P(X=7) + P(X=8) + P(X=9) + P(X=10)
10 let prob = 0;
11 for (let k = 7; k <= 10; k++) {
12     const pk = binomialPMF(k, n, p);
13     console.log('P(X=' + k + ') =', pk.toFixed(4));
14     prob += pk;
15 }
16 console.log('P(X >= 7) =', prob.toFixed(4)); // 0.3823
17
18 // 計算結果:
19 // P(X=7) = 0.2150
20 // P(X=8) = 0.1209
21 // P(X=9) = 0.0403
22 // P(X=10) = 0.0060
23 // P(X >= 7) = 0.3823 (約38.2%)
```

5.7.2 練習2：正規分布

身長が平均 170cm、標準偏差 6cm の正規分布に従うとき、175cm 以上の人の割合を求めてください。

模範解答

Listing 14 練習2の解答

```
1 const mu = 170; // 平均
2 const sigma = 6; // 標準偏差
3 const x = 175; // 求めたい値
4
5 // Zスコアに変換
6 const z = (x - mu) / sigma;
7 console.log('z =', z.toFixed(4)); // 0.8333
8
9 // P(X >= 175) = 1 - P(X < 175) = 1 - Phi(z)
10 const pBelow = ss.cumulativeStdNormalProbability(z);
11 const pAbove = 1 - pBelow;
12
13 console.log('P(X < 175) =', pBelow.toFixed(4)); // 0.7977
14 console.log('P(X >= 175) =', pAbove.toFixed(4)); // 0.2023
15
16 // 結果: 175cm以上の人は約20.2%
```

6 第5章：回帰分析

6.1 章の目的

最小二乗法による回帰分析を理解し、予測モデルの構築と評価ができるようになることを目指します。

6.2 主要な概念

単回帰分析 1つの説明変数で目的変数を予測： $y = \beta_0 + \beta_1 x + \epsilon$

重回帰分析 複数の説明変数で予測： $y = \beta_0 + \sum \beta_i x_i + \epsilon$

最小二乗法 残差平方和を最小化するパラメータ推定

残差 実測値と予測値の差 $e_i = y_i - \hat{y}_i$

決定係数 R^2 モデルの当てはまりの良さ (0~1)

自由度調整済み R^2 説明変数の数を考慮した R^2

6.3 simple-statistics の関数一覧（回帰）

関数	説明
<code>ss.linearRegression(data)</code>	単回帰の係数を計算
<code>ss.linearRegressionLine(reg)</code>	予測関数を生成
<code>ss.rSquared(data, predict)</code>	決定係数を計算

6.4 最小二乗法の公式

$$\beta_1 = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^n (x_i - \bar{x})^2}$$

$$\beta_0 = \bar{y} - \beta_1 \bar{x}$$

6.5 コード例：単回帰分析

Listing 15 単回帰分析

```

1  const x = [1, 2, 3, 4, 5, 6, 7, 8];
2  const y = [52, 58, 65, 70, 75, 82, 88, 95];
3
4  // データを [x, y] のペアに変換
5  const data = x.map((xi, i) => [xi, y[i]]);
6
7  // 回帰係数の計算
8  const reg = ss.linearRegression(data);
9  console.log('傾き:', reg.m);           // 6.04

```

```

10 console.log('切片:', reg.b);           // 45.96
11
12 // 予測関数
13 const predict = ss.linearRegressionLine(reg);
14 console.log('x=10 の予測:', predict(10)); // 106.32
15
16 // 決定係数
17 const r2 = ss.rSquared(data, predict);
18 console.log('R^2:', r2); // 0.998

```

6.6 コード例：重回帰分析（行列演算）

重回帰の係数は $\beta = (X^T X)^{-1} X^T y$ で計算します（正規方程式）。

Listing 16 重回帰分析

```

1 // サンプルデータ：賃貸物件（面積、築年数、駅距離、家賃）
2 const data = [
3   { area: 25, age: 5, distance: 3, rent: 85000 },
4   { area: 30, age: 10, distance: 5, rent: 78000 },
5   { area: 35, age: 3, distance: 7, rent: 92000 },
6   { area: 40, age: 15, distance: 2, rent: 95000 },
7   { area: 28, age: 8, distance: 10, rent: 68000 }
8 ];
9
10 // 説明変数行列 X（切片項を含む）
11 const X = data.map(d => [1, d.area, d.age, d.distance]);
12 const y = data.map(d => d.rent);
13
14 // 行列演算
15 const Xm = math.matrix(X);
16 const ym = math.matrix(y.map(v => [v]));
17
18 const XtX = math.multiply(math.transpose(Xm), Xm);
19 const Xty = math.multiply(math.transpose(Xm), ym);
20 const beta = math.multiply(math.inv(XtX), Xty);
21
22 console.log('回帰係数:', beta.toArray());
23 // [切片, 面積係数, 築年数係数, 駅距離係数]
24 // 解釈例：面積が1m2増えると家賃は約〇円増加

```

注意： $X^T X$ が特異行列（行列式が0）の場合、逆行列が計算できません。多重共線性がある場合は変数を削減するか、正則化手法の導入を検討してください。

6.7 練習問題

6.7.1 練習1：単回帰分析

以下のデータで単回帰分析を行い、回帰式と R^2 を求めてください：

```
x = [2, 4, 6, 8, 10, 12]
y = [25, 40, 55, 70, 85, 100]
```

模範解答

Listing 17 練習1の解答

```
1  const x = [2, 4, 6, 8, 10, 12];
2  const y = [25, 40, 55, 70, 85, 100];
3
4  // [x, y] ペアに変換
5  const data = x.map((xi, i) => [xi, y[i]]);
6
7  // 回帰分析
8  const reg = ss.linearRegression(data);
9  console.log('傾き (m):', reg.m);      // 7.5
10 console.log('切片 (b):', reg.b);      // 10
11
12 // 回帰式:  $y = 10 + 7.5x$ 
13
14 // 予測関数の作成
15 const predict = ss.linearRegressionLine(reg);
16
17 // 決定係数  $R^2$ 
18 const rSquared = ss.rSquared(data, predict);
19 console.log('R^2:', rSquared); // 1.0 (完全な線形関係)
20
21 // 予測例
22 console.log('x=7 のとき:', predict(7)); // 62.5
```

6.7.2 練習2：多項式回帰

以下のデータに対して、線形回帰と2次多項式回帰を行い、どちらが適切か判断してください：

```
x = [1, 2, 3, 4, 5, 6, 7, 8]
y = [1, 3, 7, 13, 21, 31, 43, 57]
```

模範解答

Listing 18 練習2の解答

```
1  const x = [1, 2, 3, 4, 5, 6, 7, 8];
2  const y = [1, 3, 7, 13, 21, 31, 43, 57];
3
4  // 線形回帰
5  const linReg = ss.linearRegression(x.map((xi, i) => [xi, y[i]]));
6  const linPredict = ss.linearRegressionLine(linReg);
7  const linR2 = ss.rSquared(x.map((xi, i) => [xi, y[i]]), linPredict);
8
9  console.log('【線形回帰】');
10 console.log('y =', linReg.b.toFixed(2), '+', linReg.m.toFixed(2), 'x');
11 console.log('R^2 =', linR2.toFixed(4)); // 0.9538
12
```

```
13 // 2次多項式回帰:  $y = b_0 + b_1x + b_2x^2$ 
14 const X = x.map(xi => [1, xi, xi * xi]);
15 const Xm = math.matrix(X);
16 const ym = math.matrix(y.map(yi => [yi]));
17
18 const XtX = math.multiply(math.transpose(Xm), Xm);
19 const Xty = math.multiply(math.transpose(Xm), ym);
20 const beta = math.multiply(math.inv(XtX), Xty).toArray().map(b => b[0]);
21
22 console.log('\n【2次多項式回帰】');
23 console.log('y =', beta[0].toFixed(2), '+', beta[1].toFixed(2), 'x +',
24             beta[2].toFixed(2), 'x^2'); // y = 1 + 0x + 1x^2
25
26 // R^2計算
27 const yMean = ss.mean(y);
28 let ssRes = 0, ssTot = 0;
29 x.forEach((xi, i) => {
30     const pred = beta[0] + beta[1] * xi + beta[2] * xi * xi;
31     ssRes += Math.pow(y[i] - pred, 2);
32     ssTot += Math.pow(y[i] - yMean, 2);
33 });
34 console.log('R^2 =', (1 - ssRes / ssTot).toFixed(4)); // 1.0000
35
36 // 結論: データは  $y = x^2$  の関係 → 2次多項式回帰が適切
```

7 第6章：高度な統計解析

7.1 章の目的

仮説検定の考え方を理解し、t 検定、カイ二乗検定、クラスタリングなどの手法を実践できるようになることを目指します。

7.2 主要な概念

帰無仮説 (H_0) 検定で否定しようとする仮説（「差がない」など）

対立仮説 (H_1) 帰無仮説が棄却されたときに採択する仮説

有意水準 (α) 第1種の過誤を許容する確率（通常 0.05）

p 値 帰無仮説の下で観測結果以上に極端な結果が得られる確率

t 検定 平均値の差の検定

カイ二乗検定 度数分布の検定（適合度、独立性）

k-means (k 平均法) 観測点（多次元）を k 個のクラスタに分け、各点が属するクラスタの重心（centroid）との距離が小さくなるように分割する非階層的クラスタリング

ckmeans (1 次元最適クラスタリング) 1 次元（数値 1 列）データを k 群に分割し、群内平方和（withinss）が最小となるように動的計画法で最適化するクラスタリング。連続量の階級化にも有用

7.3 仮説検定の手順

1. 帰無仮説 H_0 と対立仮説 H_1 を設定（両側検定か片側検定かも明記）
2. 有意水準 α を決定（通常 0.05）
3. 検定統計量（ t , χ^2 など）と、参照する分布（t 分布、 χ^2 分布など）を確認
4. 検定統計量を計算
5. 判定（次のいずれかの方法を用いる）
 - **p 値による判定**：p 値を計算し、 $p < \alpha$ なら H_0 を棄却
 - **臨界値による判定**：臨界値（分布表など）と比較し、棄却域に入れば H_0 を棄却
6. 結論を文章で述べる（「有意差がある／ない」「統計的に示唆される」など）

7.4 simple-statistics の関数一覧（検定・クラスタリング）

関数	説明
<code>ss.tTestTwoSample(a, b, diff)</code>	2 標本 t 検定（等分散を仮定）。差の帰無仮説値 <code>diff</code> （デフォルト 0）を指定可能。返り値は t 統計量（p 値ではない）。t 分布表などで p 値や臨界値判定に用いる
<code>ss.BayesianClassifier()</code>	ベイズ分類器
<code>ss.ckmeans(data, k)</code>	1 次元数値データの最適クラスタリング（Ckmeans）。動的計画法で withinss 最小化

<code>ss.kMeansCluster(points, k)</code>	多次元点（例：2次元/3次元）に対する k-means クラスタリング。ラベルと重心を返す
--	---

注意：`ckmeans` は「多次元点の k-means」ではなく、**1次元データの最適分割**として使うのが安全です。多次元の k-means は `kMeansCluster` を使用してください。

7.5 参考：クラスタリングのコード例

Listing 19 `ckmeans` と `kMeansCluster` の使い分け

```

1 // 1次元データのクラスタリング (ckmeans)
2 const x = [1, 2, 2, 3, 10, 11, 12];
3 const clustersId = ss.ckmeans(x, 2);
4 console.log(clustersId); // 似た値同士で2群に分割
5
6 // 多次元点のk-means (kMeansCluster)
7 const pts = [[0, 0], [0.2, 0.1], [5, 5], [5.2, 4.9]];
8 const km = ss.kMeansCluster(pts, 2);
9 console.log("ラベル:", km.labels); // 各点の所属クラスタ
10 console.log("重心:", km.centroids); // 各クラスタの重心

```

7.6 コード例：2標本t検定（等分散：Student）

Student の2標本t検定（等分散を仮定）を行います。`ss.tTestTwoSample` は **t統計量を返す**ことに注意してください。

Listing 20 2標本t検定（等分散）

```

1 const groupA = [85, 78, 92, 88, 76, 82, 90, 85, 79, 87];
2 const groupB = [72, 68, 75, 70, 78, 65, 74, 71, 69, 73];
3
4 // 基本統計量
5 const meanA = ss.mean(groupA);
6 const meanB = ss.mean(groupB);
7 console.log("平均A:", meanA.toFixed(2)); // 84.20
8 console.log("平均B:", meanB.toFixed(2)); // 71.50
9 console.log("平均差(A-B):", (meanA - meanB).toFixed(2)); // 12.70
10
11 // 2標本t検定（等分散：Student）
12 // tTestTwoSample は「等分散」を仮定した t 統計量を返す
13 const t = ss.tTestTwoSample(groupA, groupB, 0);
14 const df = groupA.length + groupB.length - 2; // 等分散の自由度
15
16 console.log("t 統計量:", t.toFixed(4)); // 例：6.1636
17 console.log("自由度 df:", df); // 18
18
19 // 判定（臨界値による）
20 // 両側検定・有意水準 alpha=0.05 の場合、臨界値は t_{1-alpha/2, df}

```

```

21 // 例：df=18 のとき t_crit = 2.101 (t分布表で確認)
22 const alpha = 0.05;
23 const tCrit = 2.101;
24
25 if (Math.abs(t) > tCrit) {
26   console.log("判定: |t| > t_crit より、H0 を棄却 (平均に有意差あり)");
27 } else {
28   console.log("判定: H0 を棄却できない (有意差があるとは言えない)");
29 }

```

補足：tCrit（臨界値）は df と α と両側/片側で変わります。授業では t 分布表、または外部ライブラリ（例：jstat）で p 値を求めてください。

7.7 コード例：カイ二乗適合度検定

Listing 21 カイ二乗検定

```

1 // サイコロを120回振った結果
2 const observed = [18, 22, 15, 20, 25, 20];
3 const expected = [20, 20, 20, 20, 20, 20]; // 期待度数
4
5 // カイ二乗統計量
6 let chiSq = 0;
7 observed.forEach((o, i) => {
8   chiSq += Math.pow(o - expected[i], 2) / expected[i];
9 });
10
11 console.log('chi^2:', chiSq.toFixed(4)); // 2.90
12 // 臨界値 11.07 (df=5, alpha=0.05) より小さいので棄却できない

```

7.8 練習問題

7.8.1 練習1：t検定

以下の2グループのデータに対して、平均に有意差があるか検定してください：

グループA = [78, 82, 75, 80, 85, 79, 83, 77]

グループB = [85, 88, 90, 82, 87, 91, 86, 89]

模範解答

Listing 22 練習1の解答

```

1 const groupA = [78, 82, 75, 80, 85, 79, 83, 77];
2 const groupB = [85, 88, 90, 82, 87, 91, 86, 89];
3
4 // 基本統計量
5 const meanA = ss.mean(groupA); // 79.875
6 const meanB = ss.mean(groupB); // 87.25
7 console.log('平均A:', meanA.toFixed(2));
8 console.log('平均B:', meanB.toFixed(2));

```



```
9 console.log('平均差:', (meanB - meanA).toFixed(2)); // 7.375
10
11 // t統計量を計算（等分散を仮定）
12 const t = ss.tTestTwoSample(groupA, groupB, 0);
13 const df = groupA.length + groupB.length - 2; // 14
14
15 console.log('t統計量:', t.toFixed(4)); // 約 -4.43
16 console.log('自由度:', df);
17
18 // 判定（両側検定、 $\alpha = 0.05$ ）
19 // 臨界値  $t_{0.025}(14) \approx 2.145$ 
20 const criticalT = 2.145;
21 console.log('臨界値:', criticalT);
22
23 if (Math.abs(t) > criticalT) {
24     console.log('結論: 有意差あり (H0を棄却) ');
25     console.log('グループBの方が有意に高い');
26 } else {
27     console.log('結論: 有意差があるとは言えない');
28 }
```

7.8.2 練習2：移動平均

以下のデータに対して、3期移動平均と5期移動平均を計算してください：

data = [10, 15, 12, 18, 20, 22, 25, 23, 28, 30, 32, 35]

模範解答

Listing 23 練習2の解答

```
1 const data = [10, 15, 12, 18, 20, 22, 25, 23, 28, 30, 32, 35];
2
3 // 移動平均を計算する関数
4 function movingAverage(arr, window) {
5     const result = [];
6     for (let i = window - 1; i < arr.length; i++) {
7         const slice = arr.slice(i - window + 1, i + 1);
8         result.push(ss.mean(slice));
9     }
10    return result;
11 }
12
13 // 3期移動平均
14 const ma3 = movingAverage(data, 3);
15 console.log('3期移動平均:');
16 console.log(ma3.map(v => v.toFixed(2)));
17 // [12.33, 15.00, 16.67, 20.00, 22.33, 23.33, 25.33, 27.00, 30.00, 32.33]
18
19 // 5期移動平均
20 const ma5 = movingAverage(data, 5);
21 console.log('\n5期移動平均:');
```

```
22 console.log(ma5.map(v => v.toFixed(2)));  
23 // [15.00, 17.40, 19.40, 21.60, 23.60, 25.60, 27.60, 29.60]  
24  
25 // 解釈：移動平均によりデータの変動が平滑化される  
26 // 窓サイズが大きいほど滑らかになる
```

8 関数リファレンス（総合）

8.1 mathjs 関数一覧

関数	説明
基本演算	
add, subtract, multiply, divide	四則演算
sqrt, pow, log, log10, abs	数学関数
factorial, combinations	組み合わせ
sin, cos, tan, asin, acos, atan	三角関数
線形代数	
matrix, transpose, det, inv	行列操作
dot, cross, norm	ベクトル演算
identity, zeros, ones, diag	特殊行列
lusolve, lup, qr, eigs	行列分解・解法
その他	
evaluate	数式の評価
pi, e, phi	数学定数

8.2 simple-statistics 関数一覧

関数	説明
記述統計	
mean, median, mode	代表値
min, max, sum	基本統計量
variance, sampleVariance	分散
standardDeviation, sampleStandardDeviation	標準偏差
quantile, interquartileRange	分位数
geometricMean, harmonicMean, rootMeanSquare	その他の平均
sampleSkewness, sampleKurtosis	歪度・尖度
相関・回帰	
sampleCovariance, sampleCorrelation	相関分析
linearRegression, linearRegressionLine	回帰分析
rSquared	決定係数
確率・検定	
cumulativeStdNormalProbability	正規分布 CDF
probit	正規分布分位点

zScore	標準化
tTestTwoSample	t 検定
BayesianClassifier	ベイズ分類器
ckmeans	クラスタリング

9 学習のヒント

9.1 効果的な学習方法

1. 順番に学習する：各章は前の章の知識を前提としています
2. コードを実行する：読むだけでなく、必ず手を動かしましょう
3. 練習問題に取り組む：各ノートブック（.ipynb ファイル）の末尾にある練習問題で理解を確認
4. 数式と対応させる：コードと数学的定義の対応を意識
5. 実データで試す：自分のデータで分析を実践

9.2 よくあるエラーと対処法

await の忘れ 動的インポートには `await` が必要です

配列の次元 行列演算では配列の次元に注意（ベクトルは 1 次元、行列は 2 次元）

0 除算 分散が 0 のデータでは相関係数が計算できません

サンプルサイズ 標本分散には最低 2 つのデータが必要です

9.3 発展学習

本教材の内容を習得した後は、以下のトピックへの発展が考えられます：

- 機械学習（TensorFlow.js, ml5.js）
- データ可視化（D3.js, Plotly.js）
- ベイズ統計学
- 時系列分析（ARIMA, 状態空間モデル）
- 多変量解析（因子分析、判別分析）

10 本教材の実行環境

本教材は JupyterLite と xeus-javascript を使用してブラウザ上で実行します。ここでは、これらの技術について初心者向けに解説します。

10.1 JupyterLite とは

JupyterLite は、サーバーを必要とせず、Web ブラウザだけで動作する Jupyter 環境です。通常の Jupyter Notebook や JupyterLab はサーバー（Python 環境）のインストールが必要ですが、JupyterLite ではその手間が不要です。

10.1.1 JupyterLite の特徴

- **インストール不要**：Web ブラウザとインターネット接続があれば、数秒でアクセス可能
- **軽量**：静的な Web サイトとして配信できるため、GitHub ページなどで簡単にデプロイ可能
- **オフライン対応**：一度読み込めば、一部の機能はオフラインでも利用可能
- **データの保存**：ノートブックはブラウザの IndexedDB や localStorage に保存される

10.1.2 従来の Jupyter との違い

項目	従来の Jupyter	JupyterLite
サーバー	必要（Python 環境）	不要
インストール	必要	不要
動作環境	ローカル/クラウド	ブラウザのみ
カーネル	任意（Python 等）	WebAssembly 対応のもの
データ保存	ファイルシステム	ブラウザストレージ

10.2 Xeus とは

Xeus（ゼウス）は、Jupyter カーネルを開発するための C++ フレームワークです。Jupyter カーネルとは、コードを実行してその結果を返すエンジンのことです。

通常、Jupyter カーネルを一から開発するには、Jupyter Kernel Protocol という複雑な通信プロトコルを実装する必要があります。Xeus はこのプロトコル部分を担当し、開発者が言語のインタプリタ（コード実行部分）の実装に集中できるようにします。

10.2.1 Xeus ベースのカーネル例

- **xeus-python**：Python カーネル
- **xeus-cling**：C++ カーネル
- **xeus-sql**：SQL カーネル
- **xeus-javascript**：JavaScript カーネル（本教材で使用）

10.3 xeus-javascript について

xeus-javascript は、JupyterLite 上で JavaScript コードを実行するためのカーネルです。このカーネルにより、ブラウザ上で直接 JavaScript を記述・実行し、その結果を確認できます。

10.3.1 本教材での活用

本教材では、xeus-javascript を使って以下のことを行います：

1. mathjs や simple-statistics などの数学・統計ライブラリを CDN から読み込む
2. JavaScript のコードを対話的に実行し、計算結果を即座に確認
3. 数式とコードの対応を学びながら、統計学の概念を習得

10.3.2 指導者向け情報

学習者に本教材を提供するには、xeus-javascript を組み込んだ JupyterLite 環境を事前にビルドする必要があります。詳細な手順は以下を参照してください：

- xeus-javascript リポジトリ：<https://github.com/jupyter-xeus/xeus-javascript>
- JupyterLite ドキュメント：<https://jupyterlite.readthedocs.io/>

10.4 なぜ JavaScript を使うのか

Python が統計・データ分析の主流言語である中、本教材が JavaScript を採用する理由は以下の通りです：

1. 環境構築の障壁がない：ブラウザさえあれば即座に学習開始可能
2. Web 技術との親和性：将来的なデータ可視化（D3.js など）への発展が容易
3. 非同期処理の学習：await/async など、現代的なプログラミング概念に触れられる
4. 汎用性：JavaScript はフロントエンド・バックエンド両方で使える言語

11 参考サイト

本教材の学習をさらに深めるための参考サイトを紹介します。

11.1 使用ライブラリ

mathjs 公式サイト <https://mathjs.org/>

ドキュメント、関数リファレンス、オンラインデモが充実

simple-statistics 公式サイト <https://simple-statistics.github.io/>

API ドキュメントと使用例

esm.sh <https://esm.sh/>

本教材で使用している CDN。npm パッケージを ESM 形式で提供

11.2 JavaScript 学習

MDN Web Docs <https://developer.mozilla.org/ja/docs/Web/JavaScript>

Mozilla による JavaScript の包括的なリファレンス（日本語）

JavaScript Primer <https://jsprimer.net/>

日本語で書かれた JavaScript 入門書（オンライン無料公開）

11.3 数学・統計学

統計 WEB <https://bellcurve.jp/statistics/>

統計学の基礎から応用まで日本語で解説

Khan Academy（統計学） <https://www.khanacademy.org/math/statistics-probability>

動画による統計学・確率の解説（英語）

3Blue1Brown（線形代数） <https://www.3blue1brown.com/topics/linear-algebra>

線形代数を視覚的に理解できる動画シリーズ（英語）

11.4 JupyterLite

JupyterLite 公式 <https://jupyterlite.readthedocs.io/>

ブラウザ上で動作する Jupyter 環境のドキュメント

xeus-javascript <https://github.com/jupyter-xeus/xeus-javascript>

JupyterLite 用 JavaScript カーネル。本教材の実行環境として必要。指導者はこのカーネルを組み込んだ JupyterLite 環境をビルドして学習者に提供する

Project Jupyter <https://jupyter.org/>

Jupyter プロジェクトの公式サイト

索引

CDN, 6
 ckmeans, 22

 ESM, 6

 JupyterLite, 30

 k-means, 22

 math.abs, 6
 math.add, 6
 math.combinations, 16
 math.cos, 6
 math.cross, 9
 math.det, 9
 math.diag, 9
 math.divide, 6
 math.dot, 9
 math.eigs, 9
 math.evaluate, 6
 math.factorial, 6
 math.identity, 9
 math.inv, 9
 math.log, 6
 math.log10, 6
 math.lup, 9
 math.lusolve, 9
 math.matrix, 9
 math.multiply, 6
 math.norm, 9
 math.ones, 9
 math.pow, 6
 math.qr, 9
 math.sin, 6
 math.sqrt, 6
 math.subtract, 6
 math.tan, 6
 math.transpose, 9
 math.zeros, 9
 mathjs, 4

 p 値, 22

 simple-statistics, 4
 ss.BayesianClassifier, 22
 ss.ckmeans, 22
 ss.cumulativeStdNormalProbability, 15
 ss.errorFunction, 15
 ss.geometricMean, 12
 ss.harmonicMean, 12
 ss.interquartileRange, 7
 ss.kMeansCluster, 23
 ss.linearRegression, 18
 ss.linearRegressionLine, 18
 ss.max, 6
 ss.mean, 6
 ss.median, 6
 ss.min, 6
 ss.mode, 6
 ss.probit, 15
 ss.quantile, 7
 ss.rootMeanSquare, 12

ss.rSquared, 18
 ss.sampleCorrelation, 12
 ss.sampleCovariance, 12
 ss.sampleKurtosis, 12
 ss.sampleSkewness, 12
 ss.sampleStandardDeviation, 7
 ss.sampleVariance, 7
 ss.standardDeviation, 7
 ss.tTestTwoSample, 22
 ss.variance, 7
 ss.zScore, 12

t 分布, 15
 t 検定, 22

Xeus, 30
 xeus-javascript, 31

Z スコア, 12

カイ二乗検定, 22
 ノルム, 9
 ベクトル, 9
 ポアソン分布, 15

中心極限定理, 15
 二項分布, 15
 代表値, 12

共分散, 12
 内積, 9
 動的インポート, 6
 単回帰分析, 18
 固有ベクトル, 9
 固有値, 9
 外積, 9
 対立仮説, 22
 尖度, 12
 帰無仮説, 22

散布度, 12
 最小二乗法, 18
 有意水準, 22
 正規分布, 15
 歪度, 12
 残差, 18
 決定係数, 12, 18

相関係数, 12
 確率密度関数, 15
 確率質量関数, 15
 累積分布関数, 15

自由度調整済み R^2 , 18
 行列, 9
 行列式, 9

逆行列, 9
 重回帰分析, 18